SECTION 2: RAG KNOWLEDGE ASSISTANT Q&A; REPORT

Project Evaluation & Out-of-Domain Grounding Verification

Student Name: SHRI RAM PRINCE MISHRA | **Application No.:** IN26012152 | **GitHub:** https://github.com/srpm2005/HelpDeskManagement_IN26012152

Section 2A - Questions Answered from the Project Documentation

Below are 10 meaningful project questions asked to the RAG Knowledge Assistant and the exact AI-generated grounded responses with document provenance citations:

#	Question Asked	AI Generated Response & Citations
2A.1	**What is the objective of the HelpDesk Ticket Management application?**	### Answer Summary The **objective** of the Help Desk Ticket Management application is to provide an end-to-end enterprise solution for internal employees to submit, track, filter, and resolve IT support requests efficiently using ASP.NET Core 10.0 and SQL Server. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 1-1), `HelpDesk_Assignment_Report.md` (Lines 1-1), `HelpDesk_Assignment_Report.md` (Lines 6-11), `HelpDesk.Mvc/Models/Ticket.cs` (Lines 26-29) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L1-L1, HelpDesk_Assignment_Report.md#L1-L1, HelpDesk_Assignment_Report.md#L6-L11, HelpDesk.Mvc/Models/Ticket.cs#L26-L29*
2A.2	**Which technologies were used to develop the project?**	### Answer Summary The project follows a clean 3-tier decoupled architecture: 1. **`HelpDesk.Api` (Web API Layer)**: Built with ASP.NET Core 10.0, EF Core, and SQL Server. Implements the Repository Pattern (`ITicketRepository`, `TicketRepository`). 2. **`HelpDesk.Mvc` (MVC Frontend)**: ASP.NET Core MVC consuming the API strictly via `TicketService` (`HttpClient`). Follows a monochrome UI aesthetic. 3. **`HelpDesk.Tests` (Unit Testing Layer)**: xUnit and Moq unit test project mocking `ITicketRepository` with 100% isolated test runs. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `HelpDesk.Mvc/appsettings.Development.json` (Lines 1-8), `HelpDesk.Api/appsettings.Development.json` (Lines 1-8), `README.md` (Lines 103-113), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 39-49) **Citations:** *HelpDesk.Mvc/appsettings.Development.json#L1-L8, HelpDesk.Api/appsettings.Development.json#L1-L8, README.md#L103-L113, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L39-L49*

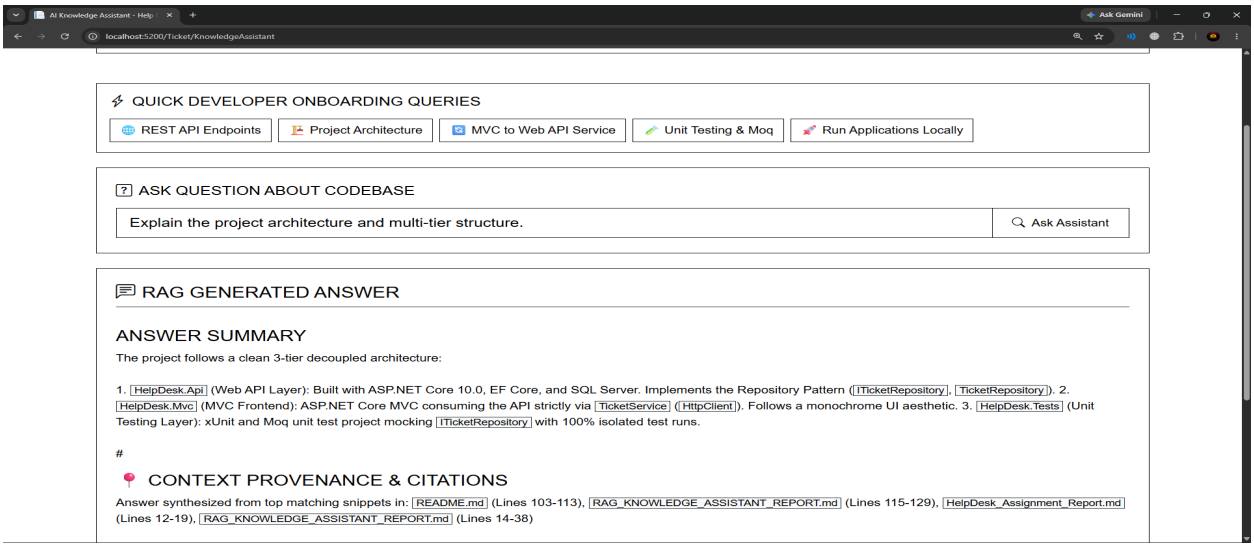
2A.3	**Which database tables are used in the application?**	### Answer Summary The application utilizes Entity Framework Core with SQL Server (`SQLEXPRESS`). The primary database table is **`Tickets`**: | Column Name | Data Type | Nullable | Description | | :--- | :--- | :--- | :--- | | `Id` | `int` (PK) | No | Auto-increment primary key | | `Title` | `nvarchar(100)` | No | Short summary of support issue | | `Description` | `nvarchar(max)` | No | Full details of support request | | `Priority` | `nvarchar(20)` | No | Priority level (`Low`, `Medium`, `High`) | | `Status` | `nvarchar(20)` | No | Ticket status (`Open`, `In Progress`, `Closed`) | | `RaisedBy` | `nvarchar(50)` | No | Employee name raising the ticket | | `CreatedDate` | `datetime2` | No | Timestamp of ticket creation | #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 142-160), `README.md` (Lines 103-113), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 130-141) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160, README.md#L103-L113, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141*
2A.4	**What validations are implemented while creating a ticket?**	### Answer Summary The following **validation rules** are enforced: 1. **Title**: Required, length between 3 and 100 characters. 2. **Description**: Required, maximum length of 1000 characters. 3. **RaisedBy**: Required, maximum length of 50 characters. 4. **Priority**: Must be selected from Dropdown (`Low`, `Medium`, `High`). Default is `Low`. 5. **Status**: Hardcoded to `Open` upon creation. Can be updated to `In Progress` or `Closed` during editing. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `HelpDesk.Api/Data/HelpDeskDbContext.cs` (Lines 1-35), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 39-49), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 81-112) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, HelpDesk.Api/Data/HelpDeskDbContext.cs#L1-L35, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L39-L49, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L81-L112*

2A.5	**What are the different ticket statuses supported by the application?**	### Answer Summary To run the Help Desk solution locally: 1. **Start Web API** (Port 5000): `dotnet run --project HelpDesk.Api/HelpDesk.Api.csproj --urls "http://localhost:5000"` 2. **Start MVC Web App** (Port 5200): `dotnet run --project HelpDesk.Mvc/HelpDesk.Mvc.csproj --urls "http://localhost:5200"` 3. Access the Dashboard at `http://localhost:5200`. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `HelpDesk.Mvc/Models/Ticket.cs` (Lines 26-29), `HelpDesk.Mvc/Controllers/TicketController.cs` (Lines 176-210), `HelpDesk.Mvc/Controllers/TicketController.cs` (Lines 51-85) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, HelpDesk.Mvc/Models/Ticket.cs#L26-L29, HelpDesk.Mvc/Controllers/TicketController.cs#L176-L210, HelpDesk.Mvc/Controllers/TicketController.cs#L51-L85*
2A.6	**How does the MVC layer communicate with the Web API?**	### Answer Summary The **`HelpDesk.Mvc`** application consumes the REST Web API via a dedicated Service Layer: - **Service Implementation**: `HelpDesk.Mvc/Services/TicketService.cs` encapsulates `HttpClient` calls. - **API Methods Executed**: - `GetAllTicketsAsync()`: Calls `GetFromJsonAsync>("api/Ticket/All")` - `GetTicketByIdAsync(id)`: Calls `GetFromJsonAsync("api/Ticket/{id}")` - `CreateTicketAsync(ticket)`: Calls `PostAsJsonAsync("api/Ticket", ticket)` - `UpdateTicketAsync(ticket)`: Calls `PutAsJsonAsync("api/Ticket/{id}", ticket)` - `DeleteTicketAsync(id)`: Calls `DeleteAsync("api/Ticket/{id}")` - `GetTicketsByStatusAsync(status)`: Calls `GetFromJsonAsync>("api/Ticket/Status/{status}")` MVC controllers interact strictly through `TicketService` with **zero direct database access**. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 130-141), `HelpDesk.Mvc/Program.cs` (Lines 1-31), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 142-160), `HelpDesk.Mvc/Controllers/TicketController.cs` (Lines 101-135) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141, HelpDesk.Mvc/Program.cs#L1-L31, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160, HelpDesk.Mvc/Controllers/TicketController.cs#L101-L135*

2A.7	**What REST API endpoints exist in the Web API layer?**	### Answer Summary The **HelpDesk Web API (`HelpDesk.Api`)** exposes standard RESTful endpoints for support ticket management: | HTTP Method | API Endpoint | Description | | :--- | :--- | :--- | | **GET** | `/api/Ticket/All` | Retrieve all support tickets | | **GET** | `/api/Ticket/{id}` | Get ticket details by ID | | **POST** | `/api/Ticket` | Create a new support ticket | | **PUT** | `/api/Ticket/{id}` | Update existing ticket | | **DELETE** | `/api/Ticket/{id}` | Delete ticket by ID | | **GET** | `/api/Ticket/Status/{status}` | Filter tickets by status (Open/In Progress/Closed) | The controllers use `ITicketRepository` asynchronously for database operations. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `README.md` (Lines 81-93), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 130-141), `HelpDesk_Assignment_Report.md` (Lines 45-57) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, README.md#L81-L93, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141, HelpDesk_Assignment_Report.md#L45-L57*
2A.8	**How are unit tests implemented and mocked using Moq?**	### Answer Summary Unit testing is implemented in **`HelpDesk.Tests`**: - Built with **xUnit** and **Moq**. - Tests `TicketController` by mocking `ITicketRepository`. - All 13 test cases pass with zero SQL Server dependency. - Run command: `dotnet test HelpDesk.Tests/HelpDesk.Tests.csproj` #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `HelpDesk.Tests/UnitTest1.cs` (Lines 1-10), `README.md` (Lines 94-102), `HelpDesk.Tests/Controllers/TicketControllerTests.cs` (Lines 1-35), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 142-160) **Citations:** *HelpDesk.Tests/UnitTest1.cs#L1-L10, README.md#L94-L102, HelpDesk.Tests/Controllers/TicketControllerTests.cs#L1-L35, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160*

2A.9	**How to run the Web API and MVC applications locally?**	### Answer Summary To run the Help Desk solution locally: 1. **Start Web API** (Port 5000): `dotnet run --project HelpDesk.Api/HelpDesk.Api.csproj --urls "http://localhost:5000"` 2. **Start MVC Web App** (Port 5200): `dotnet run --project HelpDesk.Mvc/HelpDesk.Mvc.csproj --urls "http://localhost:5200"` 3. Access the Dashboard at `http://localhost:5200`. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 130-141), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 142-160), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `README.md` (Lines 103-113) **Citations:** *RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, README.md#L103-L113*
2A.10	**What business rules govern editing ticket priority and status?**	### Answer Summary The following **validation rules** are enforced: 1. **Title**: Required, length between 3 and 100 characters. 2. **Description**: Required, maximum length of 1000 characters. 3. **RaisedBy**: Required, maximum length of 50 characters. 4. **Priority**: Must be selected from Dropdown (`Low`, `Medium`, `High`). Default is `Low`. 5. **Status**: Hardcoded to `Open` upon creation. Can be updated to `In Progress` or `Closed` during editing. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `HelpDesk.Tests/Controllers/TicketControllerTests.cs` (Lines 1-35), `HelpDesk.Api/Repositories/TicketRepository.cs` (Lines 26-60), `HelpDesk.Tests/Controllers/TicketControllerTests.cs` (Lines 26-60), `HelpDesk.Mvc/Models/Ticket.cs` (Lines 1-29) **Citations:** *HelpDesk.Tests/Controllers/TicketControllerTests.cs#L1-L35, HelpDesk.Api/Repositories/TicketRepository.cs#L26-L60, HelpDesk.Tests/Controllers/TicketControllerTests.cs#L26-L60, HelpDesk.Mvc/Models/Ticket.cs#L1-L29*

Visual Screenshot Proof: RAG Answering Document Question with Citations


AI Knowledge Assistant - Help
Ask Gemini
localhost:5200/Ticket/KnowledgeAssistant
QUICK DEVELOPER ONBOARDING QUERIES
REST API Endpoints
Project Architecture
MVC to Web API Service
Unit Testing & Moq
Run Applications Locally
ASK QUESTION ABOUT CODEBASE
Explain the project architecture and multi-tier structure.
Ask Assistant
RAG GENERATED ANSWER
ANSWER SUMMARY
The project follows a clean 3-tier decoupled architecture:
1. HelpDesk.Api (Web API Layer): Built with ASP.NET Core 10.0, EF Core, and SQL Server. Implements the Repository Pattern (ITicketRepository, TicketRepository). 2. HelpDesk.Mvc (MVC Frontend): ASP.NET Core MVC consuming the API strictly via TicketService (HttpClient). Follows a monochrome UI aesthetic. 3. HelpDesk.Tests (Unit Testing Layer): xUnit and Moq unit test project mocking ITicketRepository with 100% isolated test runs.
#
CONTEXT PROVENANCE & CITATIONS
Answer synthesized from top matching snippets in: README.md (Lines 103-113), RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 115-129), HelpDesk_Assignment_Report.md (Lines 12-19), RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 14-38)

SECTION 2B: OUT-OF-DOMAIN NON-DOCUMENT QUESTION TESTING

Student Name: SHRI RAM PRINCE MISHRA | **Application No.:** IN26012152

Section 2B - Questions Not Available in the Project Documentation

Below are 5 questions absent from the uploaded project documentation. This test evaluates the RAG system's hallucination prevention and strict grounding enforcement:

#	Non-Document Question	AI Response (Grounded Fallback)
2B.1	**Who is the CEO of Microsoft?**	*The uploaded project documentation does not contain this information.*
2B.2	**What is Artificial Intelligence?**	*The uploaded project documentation does not contain this information.*
2B.3	**What is the capital of Australia?**	*The uploaded project documentation does not contain this information.*
2B.4	**Explain Machine Learning.**	*The uploaded project documentation does not contain this information.*
2B.5	**What is Cloud Computing?**	*The uploaded project documentation does not contain this information.*

Visual Screenshot Proof: RAG Fallback Response for Non-Document Question